NODE.JS CI/CD PIPELINE WITH GITHUB ACTIONS & DOCKER

DevOps Internship Project Report

Repository	github.com/karthi0079/devops-internship
Docker Hub image	karthi2801/nodejs-demo-app:1.0
Application port	3000

1. Introduction

This project implements a basic Continuous Integration and Continuous Delivery (CI/CD) pipeline for a Node.js HTTP application. The source code is maintained in GitHub, automated testing and image publishing are described through a GitHub Actions workflow, and Docker is used to package the application into a portable container image. The repository includes the application source, a test script, Node package manifests, a Dockerfile and workflow configuration.

2. Abstract

The application uses Node.js built-in HTTP functionality to start a server on the port supplied through the PORT environment variable, defaulting to port 3000. It responds with the text “Hello from DevOps CI/CD Pipeline!”. A Node.js test script checks a simple addition function using the built-in assert module. According to the repository README, GitHub Actions checks out the code, sets up Node.js, installs dependencies with npm ci, runs npm test, authenticates to Docker Hub using repository secrets, and builds and pushes the container image when code is pushed to the main branch.

3. Tools Used

Technology	Purpose
Node.js	Runs the HTTP application and test script
Git and GitHub	Source-code version control and repository hosting
GitHub Actions	Automates CI checks and Docker image publishing
Docker	Packages the Node.js application into a container
Docker Hub	Stores and distributes the built container image
npm	Manages the Node project and runs its test command

4. Steps Involved in Building the Project

- **Application development:** Created *app.js* using Node.js's built-in *http* module. The server listens on *process.env.PORT* or port 3000 and returns a plain-text response.
- **Test implementation:** Created *app.test.js*. It defines an addition function, checks that 2 + 3 equals 5 using Node.js's built-in *assert* module, and prints a success message.
- **Package configuration:** Added *package.json* with the *npm test* script configured to execute *node app.test.js*, along with the package lock file.
- **Containerization:** Created a Dockerfile based on *node:22-alpine*, set */app* as the working directory, copied package manifests and application files, installed npm packages, exposed port 3000 and configured the container to run *node app.js*.
- **CI/CD automation:** The repository README documents a GitHub Actions workflow that checks out source code, sets up Node.js, runs *npm ci* and *npm test*, logs in to Docker Hub with GitHub repository secrets, and builds and pushes the Docker image on pushes to *main*.
- **Image publishing and execution:** The README identifies the published image as *karthi2801/nodejs-demo-app:1.0* and documents pulling it and running a container with host port 3000 mapped to container port 3000.

5. Conclusion

The repository demonstrates the core stages of a container-based CI/CD workflow: maintaining Node.js source code in GitHub, executing a test script, packaging the application with Docker, and publishing an image to Docker Hub through GitHub Actions. It provides practical exposure to source control, automated checks, Dockerfile creation, container image distribution and application execution in a container. The documented Docker image is *karthi2801/nodejs-demo-app:1.0*.

Project repository: GitHub – devops-internship | **Image:** Docker Hub – nodejs-demo-app